

Module 3 - Prompt Engineering & Security

(Two-Line Revision Notes)

Core Prompting

Clear, specific prompts produce more reliable outputs. Define the task, context, constraints, and expected format.

Zero-shot Prompting

Ask the model directly without examples. Best for straightforward tasks.

One-shot Prompting

Provide one example to guide the response. Useful when output format matters.

Few-shot Prompting

Provide multiple examples so the model learns the expected pattern. Improves consistency on complex tasks.

Chain-of-Thought

Break difficult reasoning into intermediate steps. Use when structured reasoning improves accuracy.

Prompt Chaining

Split a complex workflow into multiple prompts. Each prompt solves one sub-task before the next.

Structured Output

Ask for JSON or a predefined schema. Makes responses easy to validate and parse.

System Prompt Engineering

Use the system prompt to define the model's role, rules, and boundaries. Keep it concise and unambiguous.

Prompt Injection

Attackers try to override instructions using malicious prompts. Validate inputs and enforce business rules.

Prompt Leaking

Attackers attempt to reveal hidden prompts or instructions. Never expose system prompts or secrets.

Jailbreaking

Attempts to bypass model safety restrictions. Use layered defenses rather than relying on the model alone.

Input Validation

Check length, scope, and suspicious patterns before calling the LLM. Reject invalid requests early.

Output Filtering

Validate JSON, schema, SQL, and business rules before using model output. Never trust raw LLM responses.

Key Parameters / Design Considerations

System Prompt: defines behavior; User Prompt: task request; Temperature: response randomness; Top_p: nucleus sampling; Max Tokens: response length; Response Format: JSON/schema; Conversation History: maintains context; Validation Rules: input/output safety checks; Prompt Templates: reusable standardized prompts.